

Práctica 2

Parte 2

En la segunda parte de esta práctica, exploraremos un aspecto fundamental de los sistemas operativos en tiempo real: la creación y gestión de tareas. En un entorno de desarrollo avanzado como ESP-IDF, tenemos la capacidad de generar y administrar múltiples tareas concurrentes, lo que nos permite aprovechar al máximo el potencial de nuestros dispositivos. A diferencia de otros marcos de trabajo más simplificados, como Arduino, ESP-IDF dispone de un conjunto completo de herramientas para diseñar sistemas complejos y robustos, donde la multitarea es esencial.

En esta parte de la práctica, abordaremos cómo crear tareas en ESP-IDF y en el concepto fundamental de gestión de memoria, explorando las diferencias entre el stack y el heap. Entender estos conceptos es esencial para prevenir problemas como desbordamientos de memoria y otros errores comunes.

Sigue los siguientes pasos:

1. En el proyecto STR2024_P2, crea la configuración “parte2_1”. En su fichero main.c copia el siguiente código:

```
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_log.h"

static const char* TAG = "P2_2_1";

// Código de la tarea
void task_function(void *pvParameters)
{
    ESP_LOGI(TAG, "task_function: inicio de la tarea");
    while(1)
    {
        ESP_LOGI(TAG, "task_function: tick");
        vTaskDelay(1000 / portTICK_PERIOD_MS);
    }
}

void app_main()
{
    ESP_LOGI(TAG, "app_main: despliegue de la tarea.");
    // Función para crear tareas
    xTaskCreate(task_function, "Task", 2048, NULL, 1, NULL);
    // Un bucle, para que la ejecución no se detenga
    while(1)
    {
        vTaskDelay(1000 / portTICK_PERIOD_MS);
    }
}
```

Estos punteros están relacionados.

Lo que se pase como argumento en xTaskCreate será recibido por el código de la tarea como *pvParameters.

Esto es muy útil para pasar estructuras de control a la tarea, como veremos en la siguiente parte.

Este puntero a NULL se puede utilizar para conservar un manejador de tarea (handler), con el que se pueden hacer varias gestiones sobre la tarea desplegada.

```
}  
}
```

Apartado III: Creación de tareas y memoria de pila.

1. Aporta capturas de pantalla de la ejecución del código.
2. Propón una modificación del código para hacer uso del argumento **pvParameters*. Por ejemplo, crea una variable cualquiera en *app_main* y pásala por referencia a la tarea, donde se imprima su pantalla su valor. Aporta capturas de pantalla.
3. Busca información en la documentación de *esp-idf* sobre el argumento que se ha establecido al valor *configMINIMAL_STACK_SIZE*. Responde a las siguientes preguntas:
 - a) ¿Para qué se usa la memoria de pila (STACK)?
 - b) ¿Qué valor tiene *configMINIMAL_STACK_SIZE*?
 - c) ¿Puede modificarse esta configuración desde *menuconfig*? ¿En qué opción?
4. Propón una modificación del código para que la creación de la tarea produzca un error de desbordamiento de memoria de pila (por ejemplo, intenta reservar al inicio de la tarea un vector o una matriz muy grande). Después, modifica la asignación de memoria de pila de la tarea para que no aparezca el error. Aporta capturas de pantalla.

2. Crea la configuración “parte2_2” del proyecto. En *app_main*, introduce el siguiente código:

```
#include "freertos/FreeRTOS.h"  
#include "freertos/task.h"  
#include "esp_log.h"  
  
static const char* TAG = "P2_2_2";  
  
// Código de la tarea  
void task_function(void *pvParameters)  
{  
    ESP_LOGI(TAG, "task_function: inicio de la tarea");  
    printf("Ejecución de Task: ");  
    while(1)  
    {  
        vTaskDelay(100 / portTICK_PERIOD_MS);  
        printf(".");  
        fflush(stdout);  
    }  
}  
  
// Función auxiliar para imprimir por pantalla el estado de una tarea  
void print_task_state (TaskHandle_t handler)  
{  
    TaskStatus_t task_status;  
    vTaskGetInfo(handler, &task_status, pdTRUE, eInvalid);  
    printf("\nTask STATE: ");  
    switch (task_status.eCurrentState)  
    {  
        case eRunning:  
            printf("RUNNING\n");  
    }  
}
```

Para que este método pueda linkarse correctamente, tienes que activar la siguiente opción en *menuconfig*:

Component config ->
FreeRTOS ->
Kernel ->
configUSE_TRACE_FACILITY

```

        break;
    case eReady:
        printf("READY\n");
        break;
    case eBlocked:
        printf("BLOCKED\n");
        break;
    case eSuspended:
        printf("SUSPENDED\n");
        break;
    case eDeleted:
        printf("DELETED\n");
        break;
    default:
        printf("UNKNOWN\n");
        break;
}
}

```

```

void app_main()
{
    TaskHandle_t task_handler; // Variable de tipo manejador de tarea

    ESP_LOGI(TAG, "app_main: despliegue de la tarea.");
    // Función para crear tareas
    xTaskCreate(task_function, "Task", 2048, NULL, 1, &task_handler);

    // Tarea activa
    print_task_state(task_handler);
    vTaskDelay(3000 / portTICK_PERIOD_MS);

    // Tarea en suspensión
    vTaskSuspend(task_handler);
    vTaskDelay(100 / portTICK_PERIOD_MS);
    print_task_state(task_handler);
    vTaskDelay(3000 / portTICK_PERIOD_MS);

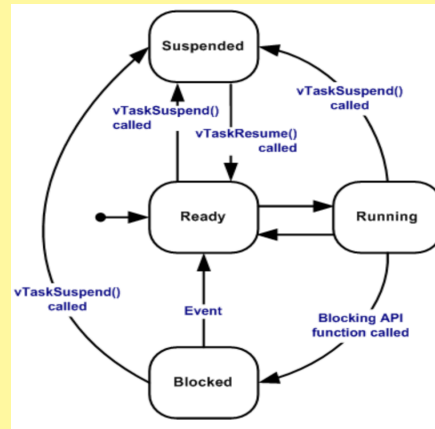
    // Reanudación
    vTaskResume(task_handler);
    vTaskDelay(100 / portTICK_PERIOD_MS);
    print_task_state(task_handler);
    vTaskDelay(3000 / portTICK_PERIOD_MS);

    // Suspensión
    vTaskSuspend(task_handler);
    vTaskDelay(100 / portTICK_PERIOD_MS);
    print_task_state(task_handler);
    vTaskDelay(3000 / portTICK_PERIOD_MS);

    // Eliminación
    vTaskDelete(task_handler);
    vTaskDelay(100 / portTICK_PERIOD_MS);
    print_task_state(task_handler);
}

```

En la función **print_task_state** puedes ver los diferentes estados que puede tener una tarea en FreeRTOS. Aquí tienes información precisa acerca de cada uno de ellos: <https://www.freertos.org/RTOS-task-states.html>



}

Apartado IV: Control y estado de las tareas.

1. Aporta captura de pantalla de la ejecución y responde a las siguientes preguntas:
 - a. Cuando se crea la tarea Task1 inicialmente, ¿en qué estado se encuentra?
 - b. ¿Por qué razón el estado de la tarea no es RUNNING? Busca información al respecto.
 - c. Cuando reanudamos la tarea, ¿ejecuta su código desde el principio? ¿por qué?
2. Implementa otra tarea Task2 en la que se imprima cada segundo el estado de la tarea Task1. Modifica app_main para que deje de imprimir el estado de Task1. Para que la nueva tarea disponga del manejador de la tarea Task1, pásasela como argumento desde app_main en el momento de su creación. Aporta capturas de pantalla de la ejecución.

3. Crea la configuración “parte2_3” del proyecto. En app_main, introduce el siguiente código:

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_log.h"

static const char* TAG = "P2_2_3";

void memory_allocation_task(void *pvParameters)
{
    while (1)
    {
        // Reservar memoria dinámica
        void *ptr = pvPortMalloc(1024);

        // Verificar si la asignación de memoria fue exitosa
        if (ptr == NULL)
        {
            ESP_LOGE(TAG, "No se pudo reservar más memoria");
            vTaskDelete(NULL); // Eliminar la tarea si no se pudo reservar memoria
        }
        ESP_LOGI(TAG, "Reserva de 1024 bytes");
        // Simular un poco de trabajo
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}

void app_main()
{
    // Crear tarea para la asignación de memoria
    xTaskCreate(memory_allocation_task, "Memory Allocation Task", 2048, NULL, 1, NULL);
    // No es necesario mantener la tarea app_main, la tarea Memory Allocation Task
    // continuará su ejecución indefinidamente
}
```

Apartado V: Memoria de bulto (HEAP)

1. *Modifica el código para que la tarea imprima una suma de la memoria que lleva reservada en cada iteración. Aporta captura de pantalla del final de la ejecución y responde a las siguientes preguntas:*
 - a. *¿Qué es la memoria HEAP y para qué se usa? Busca información al respecto.*
 - b. *¿Cuánta memoria es capaz de reservar?*
2. *Pulsa reset en la placa y, entre toda la información en pantalla generada al inicio del sistema, busca las líneas relacionadas con las particiones de memoria heap (aparece como "heap_init : ..."). Aporta captura de pantalla de estas líneas y responde a las siguientes preguntas:*
 - a. *¿Qué tipos de memoria RAM hay? Busca información al respecto en la documentación de esp-idf.*
 - b. *¿Cuánta memoria RAM **para datos** hay disponible? (suma también la memoria tipo I/D)*
 - c. *Réstale la que hemos conseguido reservar en nuestro código. ¿Cuánta memoria RAM consume mantener el sistema operativo y tus tareas en ejecución?*
 - d. *¿Qué te parece este último cálculo? Elige una respuesta:*
 - ☐ *Es mucho más de lo que esperaba*
 - ☐ *Es más de lo que esperaba*
 - ☐ *Es lo razonable*
 - ☐ *Es menos de lo que esperaba*
 - ☐ *Es mucho menos de lo que esperaba*
 - ☐ *No sabría valorar esta cantidad*